



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 04

NOMBRE COMPLETO: Carandia Lorenzo Brenda Fernanda

N° de Cuenta: 319018961

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 05

SEMESTRE 2025-2

FECHA DE ENTREGA LÍMITE: 09 de marzo de 2025

CALIFICACIÓN: _____

MODELADO JERÁRQUICO

Actividades realizadas

1.- Terminar la Grúa con:

- Cuerpo (prisma rectangular)
- Base (pirámide cuadrangular)
- 4 llantas (4 cilindros) con teclado se pueden girar las 4 llantas por separado

Bloque de código

En nuestro código Window.h

```
class Window
{
public:
    Window();
    Window(GLint windowWidth, GLint windowHeight);
    int Initialise();
    GLfloat getBufferWidth() { return bufferWidth; }
    GLfloat getBufferHeight() { return bufferHeight; }
    bool getShouldClose() {
        return glfwWindowShouldClose(mainWindow);
    }
    bool* getKeys() { return keys; }
    GLfloat getXChange();
    GLfloat getYChange();
    void swapBuffers() { return glfwSwapBuffers(mainWindow); }
    GLfloat getrotay() { return rotay; }
    GLfloat getrotax() { return rotax; }
    GLfloat getrotaz() { return rotaz; }
    GLfloat getarticulacion1() { return articulacion1; }
    GLfloat getarticulacion2() { return articulacion2; }
    GLfloat getarticulacion3() { return articulacion3; }
    GLfloat getarticulacion4() { return articulacion4; }
    GLfloat getarticulacion5() { return articulacion5; }
    GLfloat getarticulacion6() { return articulacion6; }
    GLfloat getllanta1() { return llanta1; }
    GLfloat getllanta2() { return llanta2; }
    GLfloat getllanta3() { return llanta3; }
    GLfloat getllanta4() { return llanta4; }
    ~Window();
private:
    GLFWwindow *mainWindow;
    GLint width, height;
    GLfloat rotax, rotay, rotaz, articulacion1, articulacion2, articulacion3, articulacion4, articulacion5, articulacion6;
    GLfloat llanta1, llanta2, llanta3, llanta4; // Rotaciones de las 4 llantas
    bool keys[1024];
    GLint bufferWidth, bufferHeight;
    GLfloat lastX;
    GLfloat lastY;
    GLfloat xChange;
    GLfloat yChange;
    bool mouseFirstMoved;
    void createCallbacks();
    static void ManejaTeclado(GLFWwindow* window, int key, int code, int action, int mode);
    static void ManejaMouse(GLFWwindow* window, double xPos, double yPos);
};
```

En nuestro código tanto para la class public y private declaramos las variables float que nos servirán para la rotación de las 4 llantas de la grúa, llamadas llanta1, llanta2, llanta3 y llanta4.

En nuestro Window.cpp

```
Window::Window(GLint windowWidth, GLint windowHeight)
{
    width = windowWidth;
    height = windowHeight;
    rotax = 0.0f;
    rotay = 0.0f;
    rotaz = 0.0f;
    articulacion1 = 0.0f;
    articulacion2 = 0.0f;
    articulacion3 = 0.0f;
    articulacion4 = 0.0f;
    articulacion5 = 0.0f;
    articulacion6 = 0.0f;
    llanta1 = 0.0f;
    llanta2 = 0.0f;
    llanta3 = 0.0f;
    llanta4 = 0.0f;
    for (size_t i = 0; i < 1024; i++) {
        keys[i] = 0;
    }
}

void Window::ManejaTeclado(GLFWwindow* window, int key, int code, int action, int mode)
{
    Window* theWindow = static_cast<Window*>(glfwGetWindowUserPointer(window));

    if (key == GLFW_KEY_ESCAPE && action == GLFW_PRESS)
    {
        glfwSetWindowShouldClose(window, GL_TRUE);
    }

    if (key == GLFW_KEY_E)
    {
        theWindow->rotax += 10.0;
    }
    if (key == GLFW_KEY_R)
    {
        theWindow->rotay += 10.0; // Rotar sobre el eje y 10 grados
    }
    if (key == GLFW_KEY_T)
    {
        theWindow->rotaz += 10.0;
    }
    if (key == GLFW_KEY_F)
    {
        theWindow->articulacion1 += 10.0;
    }
    if (key == GLFW_KEY_G)
    {
        theWindow->articulacion2 += 10.0;
    }
    if (key == GLFW_KEY_H)
    {
        theWindow->articulacion3 += 10.0;
    }
    if (key == GLFW_KEY_J)
    {
        theWindow->articulacion4 += 10.0;
    }
    if (key == GLFW_KEY_K)
    {
        theWindow->articulacion5 += 10.0;
    }
    if (key == GLFW_KEY_L)
    {
        theWindow->articulacion6 += 10.0;
    }

    // Rotaciones de las llantas
    if (key == GLFW_KEY_1)
    {
        theWindow->llanta1 += 10.0;
    }
    if (key == GLFW_KEY_2)
    {
        theWindow->llanta2 += 10.0;
    }
    if (key == GLFW_KEY_3)
    {
        theWindow->llanta3 += 10.0;
    }
    if (key == GLFW_KEY_4)
    {
        theWindow->llanta4 += 10.0;
    }

    if (key >= 0 && key < 1024)
    {
        if (action == GLFW_PRESS)
        {
            theWindow->keys[key] = true;
        }
        else if (action == GLFW_RELEASE)
        {
            theWindow->keys[key] = false;
        }
    }
}
```

En este caso inicializamos nuestras variables *llantaN* y posteriormente les asignamos una tecla para lograr la rotación, en este caso *llanta1* tiene asignada la tecla para el numero 1, *llanta2* la tecla con el numero 2, *llanta3* con la tecla del numero 3 y *llanta4* la tecla con el numero 4.

Bloque main

```
glm::mat4 model(1.0); //Inicializar matriz de Modelo 4x4
glm::mat4 modelaux(1.0); //Inicializar matriz de Modelo 4x4 auxiliar para la jerarquía
glm::mat4 modelbase(1.0); //Inicializar matriz de Modelo 4x4 base para la jerarquía
```

Fuera de nuestro while, inicializamos las 3 matrices que nos ayudaran con la jerarquía de nuestra grúa.

```
// SE EMPIEZA EL DIBUJO DEL BRAZO
// Cabina- Nodo Padre
model = glm::translate(model, glm::vec3(0.0f, 6.0f, -4.0f));
modelaux = model; //Se guardan los datos
model = glm::scale(model, glm::vec3(2.5f, 1.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
//La línea de proyección solo se manda una vez a menos que en tiempo de ejecución
//se programe cambio entre proyección ortogonal y perspectiva
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.392f, 0.2f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[0] -> RenderMesh(); //dibuja cubo, pirámide triangular, pirámide base cuadrangular

//Base- Nodo Hijo
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -0.8f, 0.0f));
modelbase = model; // Se guardan los datos
model = glm::scale(model, glm::vec3(3.5f, 1.0f, 2.0f));
color = glm::vec3(0.2f, 0.478f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[4] -> RenderMesh(); //dibuja cubo, pirámide triangular, pirámide base cuadrangular

//Llantas 1 - Nodo Hijo de Base
model = modelbase;
model = glm::translate(model, glm::vec3(-1.4f, -0.5f, -1.2f)); // 1. Primero trasladamos la llanta
model = glm::rotate(model, glm::radians(mainWindow.getllanta1()), glm::vec3(0.0f, 0.0f, 1.0f)); // 2. Rotación controlada por el usuario
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f)); // 3. Rotación fija para que el cilindro quede como llanta
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f)); // 4. Escalar al final si es necesario
color = glm::vec3(1.0f, 1.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[2] -> RenderMeshGeometry(); //dibuja cilindro

//Llanta 2 Nodo Hijo de Base
model = modelbase;
model = glm::translate(model, glm::vec3(-1.4f, -0.5f, 1.2f)); // 1. Primero trasladamos la llanta
model = glm::rotate(model, glm::radians(mainWindow.getllanta2()), glm::vec3(0.0f, 0.0f, 1.0f)); // 2. Rotación controlada por el usuario
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f)); // 3. Rotación fija para que el cilindro quede como llanta
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f)); // 4. Escalar al final si es necesario
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[2] -> RenderMeshGeometry(); //dibuja cilindro

//Llanta 3 Nodo Hijo de Base
model = modelbase;
model = glm::translate(model, glm::vec3(1.4f, -0.5f, -1.2f)); // 1. Primero trasladamos la llanta
model = glm::rotate(model, glm::radians(mainWindow.getllanta3()), glm::vec3(0.0f, 0.0f, 1.0f)); // 2. Rotación controlada por el usuario
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f)); // 3. Rotación fija para que el cilindro quede como llanta
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f)); // 4. Escalar al final si es necesario
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[2] -> RenderMeshGeometry(); //dibuja cilindro

//Llanta 4 Nodo Hijo de Base
model = modelbase;
model = glm::translate(model, glm::vec3(1.4f, -0.5f, 1.2f)); // 1. Primero trasladamos la llanta
model = glm::rotate(model, glm::radians(mainWindow.getllanta4()), glm::vec3(0.0f, 0.0f, 1.0f)); // 2. Rotación controlada por el usuario
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f)); // 3. Rotación fija para que el cilindro quede como llanta
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f)); // 4. Escalar al final si es necesario
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[2] -> RenderMeshGeometry(); //dibuja cilindro

//articulación 1
model = modelaux;
//rotación alrededor de la articulación que une con la cabina
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
sp.render();

//primer brazo que conecta con la cabina
// //Traslación inicial para posicionar en -2 a los objetos
//model = glm::translate(model, glm::vec3(0.0f, 0.0f, -4.0f));
//otras transformaciones para el objeto

model = glm::translate(model, glm::vec3(-1.0f, 2.0f, 0.0f));
model = glm::rotate(model, glm::radians(135.0f), glm::vec3(0.0f, 0.0f, 1.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(5.0f, 1.0f, 1.0f));

glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
//La línea de proyección solo se manda una vez a menos que en tiempo de ejecución
//se programe cambio entre proyección ortogonal y perspectiva
//glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
//glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.4f, 0.7f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[0] -> RenderMesh(); //dibuja cubo, pirámide triangular, pirámide base cuadrangular
//meshList[2] -> RenderMeshGeometry(); //dibuja las figuras geométricas cilindro y cono

//para descartar la escala que no quiere heredar se carga la información de la matrix auxiliar
model = modelaux;
//articulación 2
model = glm::translate(model, glm::vec3(2.5f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(0.0f, 0.0f, 1.0f));
modelaux = model;
//dibujar una pequeña esfera
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
sp.render();
```

```

model = modelaux;
//segundo brazo
model = glm::translate(model, glm::vec3(0.0f, -2.5f, 0.0f));

modelaux = model;
model = glm::scale(model, glm::vec3(1.0f, 5.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.5f, 0.0f, 0.5f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[8]->RenderMesh(); //dibuja cubo y pirámide triangular

model = modelaux;

//articulación 3 extremo derecho del segundo brazo
model = glm::translate(model, glm::vec3(0.0f, -2.5f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion3()), glm::vec3(0.0f, 0.0f, 1.0f));
modelaux = model;

//dibujar una pequeña esfera
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
sp.render();

// Crear instancias para completar el brazo y la cabina. Importar considerar que la cabina es el nodo padre.
//La cabina y el brazo deben de estar unidos a la cabina

// Brazo 3
model = modelaux; // Recuperar la matriz de la última articulación
model = glm::translate(model, glm::vec3(2.0f, -0.3f, -0.0f)); // Posicionar el tercer brazo
model = glm::rotate(model, glm::radians(100.0f), glm::vec3(0.0f, 0.0f, 1.0f)); // Rotar alrededor de la articulación 3
modelaux = model; // Guardar la matriz para la siguiente articulación

model = glm::scale(model, glm::vec3(1.0f, 5.0f, 1.0f)); // Escalar el cubo para que sea un brazo largo
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 0.0f); // Color azul para el tercer brazo
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); // Cambiar el color del objeto
meshList[8]->RenderMesh(); // Dibujar el cubo

// Articulación 4
model = modelaux; // Recuperar la matriz de la última articulación
model = glm::translate(model, glm::vec3(0.0f, -2.5f, 0.0f)); // Posicionar la articulación 4
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4()), glm::vec3(1.0f, 0.0f, 0.0f)); // Rotar alrededor de la articulación 4
modelaux = model; // Guardar la matriz para la canasta

// Dibujar una pequeña esfera para representar la articulación 4
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
sp.render();

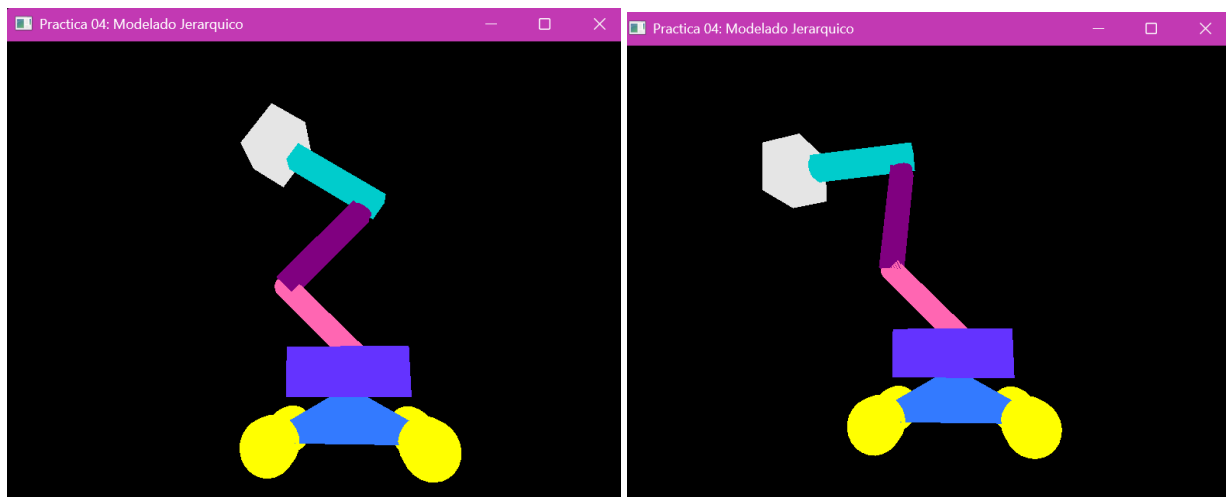
// Canasta
model = modelaux; // Recuperar la matriz de la última articulación
model = glm::translate(model, glm::vec3(0.0f, -1.0f, 0.0f)); // Posicionar la canasta
model = glm::scale(model, glm::vec3(2.5f, 2.0f, 2.5f)); // Escalar el cubo para que sea una canasta

glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.9f, 0.9f, 0.9f); // Color Gris para la canasta
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); // Cambiar el color del objeto
meshList[0]->RenderMesh(); // Dibujar el cubo

glUseProgram(0);
mainWindow.swapBuffers();
}
return 0;
}

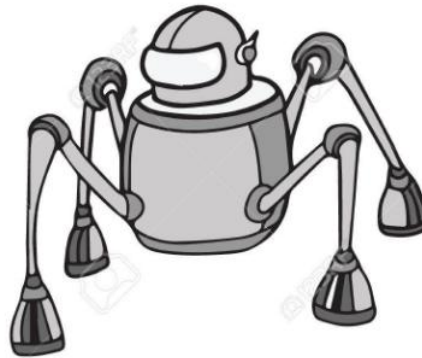
```

Ejecución de código



2.- Crear un animal robot 3d

Instanciando cubos, pirámides, cilindros, conos, esferas:
4 patas articuladas en 2 partes (con teclado se puede mover las dos articulaciones de cada pata)
cola articulada o 2 orejas articuladas. (con teclado se puede mover la cola o cada oreja independiente).



Bloque de código

- While.h

```
GLfloat getpata1() { return pata1; }
GLfloat getpata11() { return pata11; }
GLfloat getpata2() { return pata2; }
GLfloat getpata22() { return pata22; }
GLfloat getpata3() { return pata3; }
GLfloat getpata33() { return pata33; }
GLfloat getpata4() { return pata4; }
GLfloat getpata44() { return pata44; }
GLfloat getorejaD() { return orejaD; }
GLfloat getorejaI() { return orejaI; }

private:
GLFWwindow *mainWindow;
GLint width, height;
GLfloat rotax, rotay, rotaz, articulacion1, articulacion2, articulacion3, articulacion4, articulacion5, articulacion6; private:
GLfloat llanta1, llanta2, llanta3, llanta4; // Rotaciones de las 4 llantas
GLfloat pata1, pata11, pata2, pata22, pata3, pata33, pata4, pata44; // Rotaciones de las patas y articulaciones
GLfloat orejaD, orejaI; // Rotaciones de orejas
```

nuestro código tanto para la class public y private declaramos las variables float que nos servirán para la rotación de las orejas y las articulaciones de las patas

- While. cpp

```
Window::Window(GLint windowHeight, GLint windowHeight)
{
    width = windowHeight;
    height = windowHeight;
    rotax = 0.0f;
    rotay = 0.0f;
    rotaz = 0.0f;
    articulacion1 = 0.0f;
    articulacion2 = 0.0f;
    articulacion3 = 0.0f;
    articulacion4 = 0.0f;
    articulacion5 = 0.0f;
    articulacion6 = 0.0f;
    llanta1 = 0.0f;
    llanta2 = 0.0f;
    llanta3 = 0.0f;
    llanta4 = 0.0f;

    pata1 = 0.0;
    pata11 = 0.0;
    pata2 = 0.0;
    pata22 = 0.0;
    pata3 = 0.0;
    pata33 = 0.0;
    pata4 = 0.0;
    pata44 = 0.0;

    orejaD = 0.0f;
    orejaI = 0.0f;

    if (key == GLFW_KEY_Z)
    {
        theWindow->pata1 += 10.0;
    }
    if (key == GLFW_KEY_X)
    {
        theWindow->pata11 += 10.0;
    }
    if (key == GLFW_KEY_C)
    {
        theWindow->pata2 += 10.0;
    }
    if (key == GLFW_KEY_V)
    {
        theWindow->pata22 += 10.0;
    }
    if (key == GLFW_KEY_B)
    {
        theWindow->pata3 += 10.0;
    }
    if (key == GLFW_KEY_N)
    {
        theWindow->pata33 += 10.0;
    }
    if (key == GLFW_KEY_M)
    {
        theWindow->pata4 += 10.0;
    }
    if (key == GLFW_KEY_P)
    {
        theWindow->pata44 += 10.0;
    }
    if (key == GLFW_KEY_0)
    {
        theWindow->orejaD += 10.0;
    }
    if (key == GLFW_KEY_9)
    {
        theWindow->orejaI += 10.0;
    }
}
```

En este caso inicializamos nuestras variables pataN, orejaD y orejaI; posteriormente les asignamos una tecla para lograr la rotación, en este caso utilizamos las teclas inferiores del teclado para las articulaciones y los números cero y nueve para las orejas.

- Bloque main

```
glm::mat4 model(1.0); //Inicializar matriz de Modelo 4x4
glm::mat4 modelaux(1.0); //Inicializar matriz de Modelo 4x4 auxiliar para la jerarquía
glm::mat4 modelart1(1.0); //Inicializar matriz de Modelo 4x4 base para la jerarquía
glm::mat4 modelart2(1.0); //Inicializar matriz de Modelo 4x4 base para la jerarquía
glm::mat4 modelart3(1.0); //Inicializar matriz de Modelo 4x4 base para la jerarquía
glm::mat4 modelart4(1.0); //Inicializar matriz de Modelo 4x4 base para la jerarquía
glm::mat4 modelorel1(1.0); //Inicializar matriz de Modelo 4x4 base para la jerarquía
glm::mat4 modelore2(1.0); //Inicializar matriz de Modelo 4x4 base para la jerarquía
```

Fuera de nuestro while, inicializamos las 7 matrices que nos ayudaran con la jerarquía de nuestra grúa, una por cada pata y oreja que se tiene, puesto que cada pata debe tener como nodo padre al cuerpo, por lo que hay que regresar a la posición de base.

- Creación del cuerpo de la araña

```
/* Creando el cuerpo de la araña */

//para reiniciar la matriz de modelo con valor de la matriz identidad
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 6.0f, -4.0f));
modelaux = model; //Se guardan los datos
model = glm::scale(model, glm::vec3(2.5f, 3.5f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
/*la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
se programe cambio entre proyección ortogonal y perspectiva*/
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.6f, 0.6f, 0.6f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[2] -> RenderMeshGeometry(); //dibuja cilindro

//Creando la cabeza
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 2.0f, 0.0f));
model = glm::scale(model, glm::vec3(1.4f, 1.6f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.8f, 0.8f, 0.8f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
sp.render();

//Creando orejas
//Derecha
model = modelaux;
model = glm::translate(model, glm::vec3(1.28f, 2.6f, 0.4));
model = glm::rotate(model, glm::radians(mainWindow.getorejaD()), glm::vec3(1.0f, 0.0f, 0.0f)); // Rotar alrededor de X
modelorel1 = model;
model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(-235.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[1] -> RenderMesh(); // Dibujar piramide Triangular

model = modelorel1;
model = glm::translate(model, glm::vec3(0.2f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.2f, 0.2f, 0.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
sp.render(); //Dibujar esfera
```



```

//Izquierda
model = modelaux;
model = glm::translate(model, glm::vec3(-1.28f, 2.6f, 0.4));
model = glm::rotate(model, glm::radians(mainWindow.getorejaI()), glm::vec3(1.0f, 0.0f, 0.0f)); // Rotar alrededor x
modelore2 = model;
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(-235.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[1]-->RenderMesh(); // Dibujar piramide

model = modelore2;
model = glm::translate(model, glm::vec3(-0.2f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.25f, 0.25f, 0.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
sp.render();

//OJOS
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 2.6f, 1.0));
model = glm::scale(model, glm::vec3(1.0f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
sp.render(); //Dibujar esfera

```

```

//CREANDO ARTICULACIONES
//ARTICULACION 1
model = modelaux;
model = glm::translate(model, glm::vec3(-2.0f, -0.5f, 1.4f));
model = glm::rotate(model, glm::radians(mainWindow.getpatalI()), glm::vec3(0.0f, 1.0f, 0.0f)); // Rotar alrededor de Y
modelart1 = model;
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
sp.render(); //Dibujar esfera

//Extremidad 1 - Patal
model = modelart1; //Recupero la informacion de la articulacion 1
model = glm::translate(model, glm::vec3(-1.0f, 1.0f, 0.2f));
model = glm::rotate(model, glm::radians(135.0f), glm::vec3(0.0f, 0.0f, 1.0f));
modelart1 = model;
model = glm::scale(model, glm::vec3(3.0f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.4f, 0.4f, 0.4f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[0]-->RenderMesh(); // Dibujar e cubo

//Segunda Articulacion - Pata 1
model = modelart1;
model = glm::translate(model, glm::vec3(1.6f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getpatalI()), glm::vec3(0.0f, 0.0f, 1.0f)); // Rotar alrededor de Z
modelart1 = model;
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
sp.render(); // Dibujar esfera

//Segunda Extremidad - Pata 1
model = modelart1;
model = glm::translate(model, glm::vec3(-1.0f, 1.6f, 0.0f));
model = glm::rotate(model, glm::radians(125.0f), glm::vec3(0.0f, 0.0f, 1.0f));
modelart1 = model;
model = glm::scale(model, glm::vec3(3.5f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.4f, 0.4f, 0.4f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[0]-->RenderMesh(); // Dibujar e cubo

// Articulacion Extra - Pata 1
model = modelart1;
model = glm::translate(model, glm::vec3(1.6f, 0.0f, 0.0f));
modelart1 = model;
model = glm::scale(model, glm::vec3(0.35f, 0.35f, 0.35f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
sp.render(); // Dibujar esfera

//Pie 1
model = modelart1;
model = glm::translate(model, glm::vec3(0.8f, 0.1f, 0.0f));
model = glm::rotate(model, glm::radians(100.0f), glm::vec3(0.0f, 0.0f, 1.0f)); //
modelart1 = model;
model = glm::scale(model, glm::vec3(0.5f, 2.0f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.545, 0.27, 0.07);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[3]-->RenderMeshGeometry(); // Dibujar cono

```

```

//ARTICULACION 2
model = modelaux;
model = glm::translate(model, glm::vec3(2.0f, -0.5f, 1.4f));
model = glm::rotate(model, glm::radians(mainWindow.getpata2()), glm::vec3(0.0f, 1.0f, 0.0f)); // Rotar alrededor de Y
modelart2 = model;
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
sp.render(); //Dibujo esfera

//Extremidad - Pata 2
model = modelart2; //Recupero la informacion de la articulacion 2
model = glm::translate(model, glm::vec3(1.0f, 1.0f, 0.2f));
model = glm::rotate(model, glm::radians(-135.0f), glm::vec3(0.0f, 0.0f, 1.0f));
modelart2 = model;
model = glm::scale(model, glm::vec3(3.0f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.4f, 0.4f, 0.4f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[0]->RenderMesh(); // Dibujar e cubo

//Segundo Articulacion Pata 2
model = modelart2;
model = glm::translate(model, glm::vec3(-1.6f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getpata22()), glm::vec3(0.0f, 0.0f, 1.0f)); // Rotar alrededor de Z
modelart2 = model;
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
sp.render(); // Dibujar esfera

//Segunda Extremidad Pata 2
model = modelart2;
model = glm::translate(model, glm::vec3(1.0f, 1.6f, 0.0f));
model = glm::rotate(model, glm::radians(-125.0f), glm::vec3(0.0f, 0.0f, 1.0f));
modelart2 = model;
model = glm::scale(model, glm::vec3(3.5f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.4f, 0.4f, 0.4f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[0]->RenderMesh(); // Dibujar e cubo

// Articulacion Extra pata 2
model = modelart2;
model = glm::translate(model, glm::vec3(-1.6f, 0.0f, 0.0f));
modelart2 = model;
model = glm::scale(model, glm::vec3(0.35f, 0.35f, 0.35f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
sp.render(); // Dibujar esfera

//Pie 2
model = modelart2;
model = glm::translate(model, glm::vec3(-0.8f, 0.1f, 0.0f));
model = glm::rotate(model, glm::radians(-100.0f), glm::vec3(0.0f, 0.0f, 1.0f));
modelart2 = model;
model = glm::scale(model, glm::vec3(0.5f, 2.0f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.545, 0.27, 0.07);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[3]->RenderMeshGeometry(); // Dibujar Cono

```

```

//ARTICULACION 3
model = modelaux;
model = glm::translate(model, glm::vec3(-1.8f, -0.5f, -1.5f));
model = glm::rotate(model, glm::radians(mainWindow.getpata3()), glm::vec3(0.0f, 1.0f, 0.0f)); // Rotar alrededor de Y
modelart3 = model;
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
sp.render(); //Dibuja esfera

//Extremidad - Pata 3
model = modelart3; //Recupero la informacion de la articulacion 3
model = glm::translate(model, glm::vec3(-1.0f, 1.0f, 0.0f));
model = glm::rotate(model, glm::radians(120.0f), glm::vec3(0.0f, 0.0f, 1.0f));
modelart3 = model;
model = glm::scale(model, glm::vec3(3.0f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.4f, 0.4f, 0.4f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[0] -> RenderMesh(); // Dibujar e cubo

//Segunda Articulacion - Pata 3
model = modelart3;
model = glm::translate(model, glm::vec3(1.6f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getpata33()), glm::vec3(0.0f, 0.0f, 1.0f)); // Rotar alrededor de Z
modelart3 = model;
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
sp.render(); // Dibujar esfera

//Segunda Extremidad Pata 3
model = modelart3;
model = glm::translate(model, glm::vec3(-1.6f, 1.0f, 0.0f));
model = glm::rotate(model, glm::radians(145.0f), glm::vec3(0.0f, 0.0f, 1.0f));
modelart3 = model;
model = glm::scale(model, glm::vec3(3.5f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.4f, 0.4f, 0.4f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[0] -> RenderMesh(); // Dibujar e cubo

//Articulacion Extra pata 3
model = modelart3;
model = glm::translate(model, glm::vec3(1.6f, 0.0f, 0.0f));
modelart3 = model;
model = glm::scale(model, glm::vec3(0.35f, 0.35f, 0.35f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
sp.render(); // Dibujar esfera

//Pie 3
model = modelart3;
model = glm::translate(model, glm::vec3(0.8f, 0.1f, 0.0f));
model = glm::rotate(model, glm::radians(100.0f), glm::vec3(0.0f, 0.0f, 1.0f));
modelart3 = model;
model = glm::scale(model, glm::vec3(0.5f, 2.0f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.545, 0.27, 0.07);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[3] -> RenderMeshGeometry(); // Dibujar cono

```

```

//ARTICULACION 4
model = modelaux;
model = glm::translate(model, glm::vec3(1.8f, -0.5f, -1.5f));
model = glm::rotate(model, glm::radians(mainWindow.getpata4()), glm::vec3(0.0f, 1.0f, 0.0f)); // Rotar alrededor de Y
modelart4 = model;
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
sp.render(); //Dibujar esfera

//Extremidad Pata 4
model = modelart4; //Recupero la informacion de la articulacion 3
model = glm::translate(model, glm::vec3(1.0f, 1.0f, 0.0f));
model = glm::rotate(model, glm::radians(-120.0f), glm::vec3(0.0f, 0.0f, 1.0f));
modelart4 = model;
model = glm::scale(model, glm::vec3(3.0f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.4f, 0.4f, 0.4f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[0] -> RenderMesh(); // Dibujar e cubo

//Segunda Articulacion pata 4
model = modelart4;
model = glm::translate(model, glm::vec3(-1.6f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getpata4()), glm::vec3(0.0f, 0.0f, 1.0f)); // Rotar alrededor de Z
modelart4 = model;
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
sp.render(); // Dibujar esfera

//Segunda Extremidad Pata 4
model = modelart4;
model = glm::translate(model, glm::vec3(1.6f, 1.2f, 0.0f));
model = glm::rotate(model, glm::radians(-145.0f), glm::vec3(0.0f, 0.0f, 1.0f));
modelart4 = model;
model = glm::scale(model, glm::vec3(3.5f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.4f, 0.4f, 0.4f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[0] -> RenderMesh(); // Dibujar e cubo

//Articulacion Extra pata 4
model = modelart4;
model = glm::translate(model, glm::vec3(-1.6f, 0.0f, 0.0f));
modelart4 = model;
model = glm::scale(model, glm::vec3(0.35f, 0.35f, 0.35f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
sp.render(); // Dibujar esfera

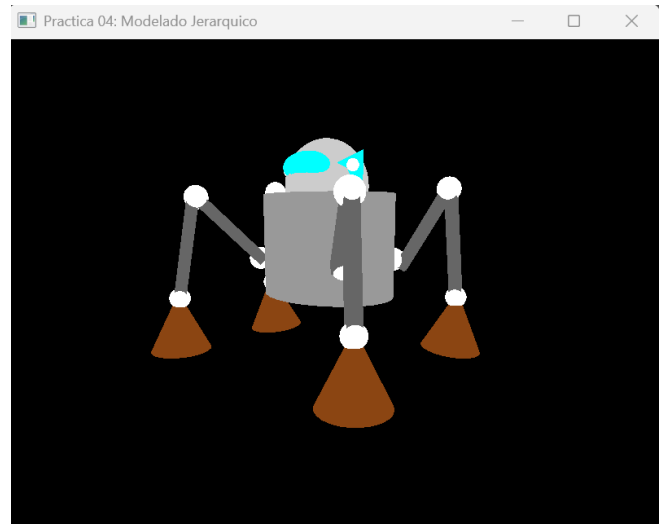
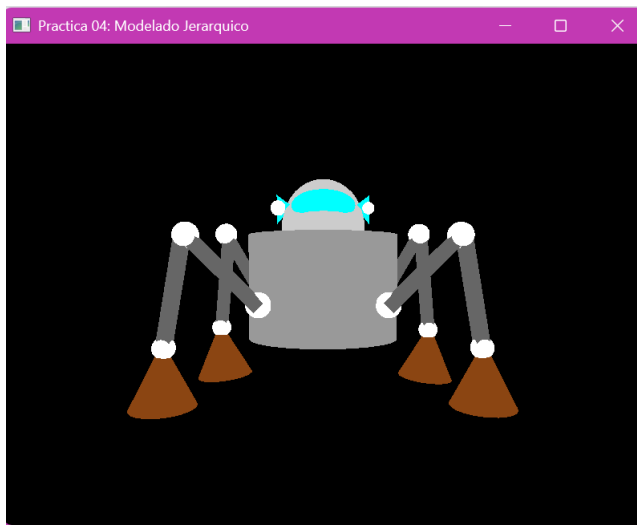
//Pie 4
model = modelart4;
model = glm::translate(model, glm::vec3(-0.8f, 0.1f, 0.0f));
model = glm::rotate(model, glm::radians(-100.0f), glm::vec3(0.0f, 0.0f, 1.0f)); // Rotar alrededor de la articulaci3n 3
modelart4 = model;
model = glm::scale(model, glm::vec3(0.5f, 2.0f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.545, 0.27, 0.07);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[3] -> RenderMeshGeometry(); // Dibujar cono

glUseProgram(0);
mainWindow.swapBuffers();

```

En esta parte del código, ejecutamos transformaciones para crear el cuerpo de la araña, utilizando cilindros para el cuerpo, cubos para las extremidades, esferas para las articulaciones, pirámides triangulares para las orejas y conos para los pies. Cada una de estas figuras debidamente trasladada. Utilice matrices auxiliares para cada extremidad, puesto que como nodo padre tenía que ser el cuerpo y las matrices modelartN me ayudaban a volver a la posición inicial y así poder hacerlo con cada extremidad. Lo mismo con las orejas, pues se dividían en dos partes.

- Ejecución de código



- Problemas presentados.

En esta practica no se me presentaron problemas a la hora de ejecutar el código.

- Conclusiones

Estos ejercicios fueron de gran ayuda para entender cómo funciona el modelado jerárquico, puesto que se veía cómo iban heredando sus características con la matriz auxiliar, se guardaban los datos. Los ejercicios me parecieron de la complejidad adecuada, al principio me costó un poco entender como es que funcionaba la jerarquía, pero con el video del profesor y su explicación pude comprenderlo con mayor claridad.

- Bibliografía

- 🚦 Códigos de colores HTML. (s. f.). HTML Color Codes.
<https://htmlcolorcodes.com/es/>
- 🚦 Martín, D. (2018). *Informática gráfica con OpenGL 4*. Universidad de Granada.
https://calipso.ugr.es/dmartin/pdfs/teach_books/informatica_grafica_con_opengl4_5_v1.pdf